AI Assistance Disclosure. ChatGPT was used to polish the English and to assist with writing and debugging the accompanying Python code. The proof and mathematical exposition were rewritten and verified by the author. This statement was drafted by ChatGPT and reviewed by the author.

Implementation and Analysis of Wavelet Trees for Rank, Select, and FM Index Pattern Counting

DING MINZHAO

July 25, 2026

Abstract

This report studies the Wavelet Tree theorem presented in the course. A Wavelet Tree represents a string through a hierarchy of bit vectors and supports access, rank, and select queries. The report explains its construction and proves that the bit vectors require $(n + o(n)) \log_2 \sigma + O(\sigma \log n)$ bits while each query takes $O(\log \sigma)$ time. A simplified Python implementation is then developed with explicit prefix sum arrays and position arrays. The implementation is used to store the Burrows Wheeler transform and to support pattern counting with the FM index. Correctness tests confirm that all answers agree with direct methods. The experiments also show that Wavelet Tree query times remain stable for a fixed alphabet and that FM index counting scales more efficiently than direct scanning.

Contents

1 Introduction **2**

2 Preliminaries and Construction **2**
2.1 Wavelet Tree Construction 3

3 Query Algorithms and Analysis **4**
3.1 Space and Query Complexity 5

4 Simplified Implementation **7**

5 FM Index and Experiments **7**
5.1 Experimental Results 9

6 Conclusion **11**

1 Introduction

Modern applications often need to store and process very large strings. Typical examples include genomic sequences, collections of Web pages, and large text databases. In such applications, a conventional index may occupy much more space than the original text. For example, a suffix array for a string of length n stores one integer for each suffix and therefore requires about $n \log n$ bits. In comparison, the string itself requires $n \log \sigma$ bits when the alphabet size is σ.

Succinct data structures aim to reduce this gap. Their goal is to represent data using space close to the information theoretic lower bound while still supporting efficient queries. Bit vectors with rank and select operations are basic components of many succinct data structures. These operations allow a compact representation to be searched and navigated without storing a large pointer based structure.

The wavelet tree applies these bit vector techniques to strings over a finite alphabet. It recursively divides the alphabet into smaller groups and stores a bit vector at each internal node. The resulting structure supports the operations access(T, i), rank$_c(T, i)$, and select$_c(T, j)$. If rank and select on each bit vector take constant time, each of these operations can be completed in $O(\log \sigma)$ time.

Wavelet trees also play an important role in compressed text indexing. The FM index stores the Burrows Wheeler transform of a text and uses backward search to count pattern occurrences [2]. Each step of backward search requires rank queries on the transformed string. A wavelet tree can therefore store the transformed string and provide the operations required by the search algorithm.

This report studies the wavelet tree theorem presented in the course. It first introduces the required bit vector operations and explains the construction of the wavelet tree. It then proves the space bound and the query time bound. A simplified Python implementation is also presented. Each node stores an explicit prefix sum array, which makes the algorithms easy to inspect and test. Finally, the implementation is used for FM index pattern counting, and its results are compared with direct scanning.

2 Preliminaries and Construction

Let Σ be a finite ordered alphabet of size σ. A string T of length n over Σ is written as

$$T = T[1]T[2] \cdots T[n], \qquad T[i] \in \Sigma. \tag{1}$$

This report uses one based indexing. The alphabet contains a unique terminator \$, which appears only at the end of the text and is smaller than every other character. This assumption guarantees that all suffixes of the text have distinct lexicographic orders.

A bit vector B of length m is a sequence $B[1]B[2] \cdots B[m]$, where each entry belongs to $\{0, 1\}$. For $b \in \{0, 1\}$, the rank operation counts the number of occurrences of b in a prefix of B:

$$\text{rank}_b(B, i) = |\{k \mid 1 \le k \le i,\ B[k] = b\}|. \tag{2}$$

We define $\text{rank}_b(B, 0) = 0$. This convention is useful when a query range begins at the first position.

The select operation returns the position of a specified occurrence of b:

$$\text{select}_b(B, j) = \min\{i \mid \text{rank}_b(B, i) = j\}. \tag{3}$$

The value is defined when B contains at least j occurrences of b. For example, if $B =$ `100101`, then $\text{rank}_1(B, 4) = 2$ and $\text{select}_1(B, 3) = 6$.

The same operations can be defined for a string over a general alphabet. For $c \in \Sigma$, the three queries considered in this report are

$$\begin{aligned} \text{access}(T, i) &= T[i], \\ \text{rank}_c(T, i) &= |\{k \mid 1 \le k \le i,\ T[k] = c\}|, \\ \text{select}_c(T, j) &= \min\{i \mid \text{rank}_c(T, i) = j\}. \end{aligned} \tag{4}$$

The wavelet tree reduces these string queries to rank and select queries on bit vectors. Following the course setting, we assume that σ is a power of two. Each character $c \in \Sigma$ is assigned a distinct binary code $C(c)$ of length $h = \log_2 \sigma$. The wavelet tree therefore has h levels.

The theoretical analysis assumes that rank and select on each bit vector take constant time. The course shows that a bit vector of length m can support these operations in constant time using $m + O(m \log\log m / \log m)$ bits. Our Python implementation uses explicit arrays, so its memory use is larger. The query paths through the wavelet tree remain the same.

The running example in this report is $T = \texttt{acagcagg\$}$, with $\Sigma = \{\$, a, c, g\}$. Since $\sigma = 4$, each character can be represented by a binary code of length two. This example will be used for the construction, query algorithms, and FM index.

2.1 Wavelet Tree Construction

A wavelet tree is a complete binary tree whose leaves correspond to the characters of the alphabet. Since σ is assumed to be a power of two, every character has a binary code of the same length $h = \log_2 \sigma$. Each internal node is associated with a subsequence of the original string and stores a bit vector. The subsequence is used to describe the construction and is not stored explicitly in the theoretical representation.

Let v be a node at depth d, where the root has depth zero. Let T_v denote the conceptual subsequence associated with v. The bit vector B_v records the next code bit of every character in T_v:

$$B_v[i] = \begin{cases} 0, & \text{if the next bit of } C(T_v[i]) \text{ is } 0, \\ 1, & \text{if the next bit of } C(T_v[i]) \text{ is } 1. \end{cases} \tag{5}$$

The characters whose next code bit is zero form the subsequence of the left child. The remaining characters form the subsequence of the right child. Their relative order is preserved during this partition. The same procedure is then applied recursively to both children. The recursion ends when a node corresponds to a single character.

For the running example, the alphabet is $\Sigma = \{\$, a, c, g\}$. We assign the codes shown in table 1.

Table 1: Binary codes used for the running example

Character	Binary code
`$`	`00`
`a`	`01`
`c`	`10`
`g`	`11`

Consider the string $T = \texttt{acagcagg\$}$. At the root, the first code bit of each character produces $B_\varepsilon = \texttt{010110110}$. The characters marked by zero form $T_0 = \texttt{aaa\$}$, while the characters marked by one form $T_1 = \texttt{cgcgg}$.

At the next level, the second code bits are stored. The left child stores $B_0 = \texttt{1110}$, and the right child stores $B_1 = \texttt{01011}$. Each resulting child now corresponds to one character. The complete construction is shown in figure 1.

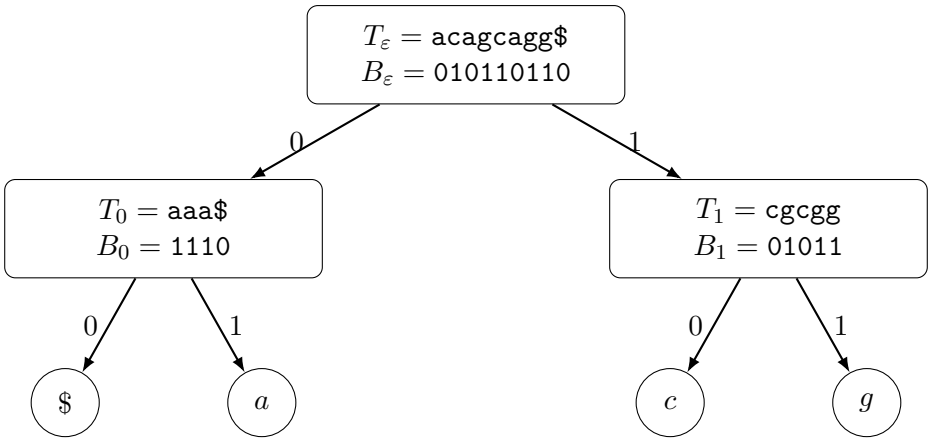


Figure 1: Wavelet tree construction for $T =$ acagcagg$

The tree has $h = \log_2 \sigma$ levels of bit vectors. At every level, each character of the original string appears in exactly one node. Therefore, the total length of all bit vectors at one level is n. This property will be used later in the space analysis.

The sequence of edge labels on the path from the root to a leaf is the binary code of the character stored at that leaf. The bit vectors also preserve the position of each character within the corresponding subsequence. These two properties allow access, rank, and select queries to move between consecutive levels of the tree.

3 Query Algorithms and Analysis

The wavelet tree supports access, rank, and select by translating a position between adjacent levels. A downward step from a node v uses the bit b associated with the selected branch. If the current position is i, its position in the child sequence is

$$i' = \text{rank}_b(B_v, i). \tag{6}$$

This equation preserves the relative order of all characters sent to the same child. It is used by both access and rank.

Access. To compute access(T, i), the algorithm begins at the root. At each internal node, it reads the bit stored at the current position. This bit determines the next child. The position is then updated using equation (6). When the traversal reaches a leaf, the character represented by that leaf is returned.

Algorithm 1 Access query on a wavelet tree

Require: A wavelet tree for T and a position i

1: $v \leftarrow$ root
2: **while** v is an internal node **do**
3: $\quad b \leftarrow B_v[i]$
4: $\quad i \leftarrow \text{rank}_b(B_v, i)$
5: $\quad v \leftarrow \text{child}(v, b)$
6: **end while**
7: **return** the character represented by v

For example, consider access$(T, 6)$ for $T =$ acagcagg$. At the root, $B_\varepsilon[6] = 0$, so the search moves to the left child. The new position is $\text{rank}_0(B_\varepsilon, 6) = 3$. At that child, $B_0[3] = 1$, so the search reaches the leaf for a. Therefore, access$(T, 6) = a$.

Rank. To compute $\text{rank}_c(T, i)$, the algorithm follows the code of c from the root to its leaf. At depth d, it reads the bit $C(c)[d]$ and updates the current prefix length with a rank query. After the final level, the current value equals the number of occurrences of c in $T[1..i]$.

Algorithm 2 Rank query on a wavelet tree

Require: A wavelet tree for T, a character c, and a position i

```
v ← root
for d ← 1 to h do
    b ← C(c)[d]
    i ← rank_b(B_v, i)
    v ← child(v, b)
end for
return i
```

For example, the code of g is `11`. To compute $\text{rank}_g(T, 8)$, the first step gives $\text{rank}_1(B_\varepsilon, 8) = 5$. The second step gives $\text{rank}_1(B_1, 5) = 3$. Hence, $\text{rank}_g(T, 8) = 3$.

Select. Select follows the path in the opposite direction. It begins at the leaf for c and moves toward the root. Suppose that the current occurrence has position j in a child sequence. If the corresponding branch is labeled b, its position in the parent sequence is

$$j' = \text{select}_b(B_v, j), \tag{7}$$

where B_v is the bit vector stored at the parent. Repeating this step for every code bit gives the position in the original string.

Algorithm 3 Select query on a wavelet tree

Require: A wavelet tree for T, a character c, and an occurrence j

```
v ← leaf(c)
for d ← h down to 1 do
    b ← C(c)[d]
    v ← parent(v)
    j ← select_b(B_v, j)
end for
return j
```

For example, the code of c is `10`. To compute $\text{select}_c(T, 2)$, the search begins at the leaf for c. The second zero in $B_1 = \texttt{01011}$ is at position 3. The third one in $B_\varepsilon = \texttt{010110110}$ is at position 5. Thus, $\text{select}_c(T, 2) = 5$.

Each algorithm visits one internal node for every bit in the character code. Since the code length is $h = \log_2 \sigma$, each query performs $O(\log \sigma)$ bit vector operations. A complete proof of this time bound is given in section 3.1.

3.1 Space and Query Complexity

This subsection proves the main theorem studied in this report. We assume that the alphabet size σ is a power of two. We use the succinct bit vector result established earlier in the course: a bit vector of length m can support rank and select in constant time using $m + o(m)$ bits.

Theorem 3.1 (Wavelet tree)**.** *Let T be a string of length n over an alphabet of size σ. A wavelet tree represents T using*

$$(n + o(n)) \log_2 \sigma + O(\sigma \log n) \tag{8}$$

bits. It supports access, rank, and select queries in $O(\log \sigma)$ time.

Proof. Let $h = \log_2 \sigma$ be the height of the wavelet tree. Consider all nodes at a fixed depth d, where $0 \leq d < h$. The subsequences stored at these nodes form a partition of the original string. Every character of T appears in exactly one subsequence at this depth. Since each character contributes one bit, the total length of the bit vectors at depth d is

$$\sum_{v \in V_d} |B_v| = n, \tag{9}$$

where V_d is the set of nodes at depth d.

At each depth, all bit vectors are concatenated from left to right into one bit vector $B^{(d)}$. By equation (9), the resulting vector has length n. A succinct bit vector representation supports rank and select on $B^{(d)}$ in constant time using $n + o(n)$ bits.

For each node v, let its bit vector occupy the interval $[s_v, e_v]$ inside $B^{(d)}$. A local rank query can be obtained from two rank queries on the concatenated vector:

$$\text{rank}_b(B_v, i) = \text{rank}_b(B^{(d)}, s_v + i - 1) - \text{rank}_b(B^{(d)}, s_v - 1). \tag{10}$$

Similarly, a local select query can be obtained by

$$\text{select}_b(B_v, j) = \text{select}_b\left(B^{(d)}, \text{rank}_b(B^{(d)}, s_v - 1) + j\right) - s_v + 1. \tag{11}$$

Thus, rank and select on each node vector still take constant time.

There are $h = \log_2 \sigma$ levels of bit vectors. The total space required for the concatenated vectors is therefore

$$(n + o(n)) \log_2 \sigma. \tag{12}$$

The wavelet tree contains σ leaves and $\sigma - 1$ internal nodes. For each node, the data structure stores the range of its bit vector inside the concatenated vector. Each range boundary requires $O(\log n)$ bits. The total space for these boundaries and the tree structure is therefore $O(\sigma \log n)$ bits.

Combining this information with equation (12) gives the space bound in equation (8).

We next consider the query time. An access query begins at the root and visits one node at each depth. At a node v, the algorithm reads one bit of B_v. This bit determines the next child. One rank query then converts the current position into the corresponding position in the child sequence.

The partition used during construction preserves the relative order of the characters. The converted position therefore refers to the same character at the next level. After h steps, the traversal reaches the leaf that represents $T[i]$.

A rank query follows the binary code of the queried character. After the first d code bits have been processed, the current position equals the number of characters in the queried prefix whose codes begin with these d bits. After all h bits have been processed, only the queried character has the complete code. The final position is therefore the required rank.

A select query follows the same path from the leaf to the root. At each level, one select operation converts a position in the child sequence into its position in the parent sequence. After h steps, the algorithm reaches the position of the required occurrence in the original string.

Each query performs a constant amount of work at every level. Since the height of the tree is $h = \log_2 \sigma$, access, rank, and select all take $O(\log \sigma)$ time. □

The theorem describes the theoretical representation used in the course. Its space bound depends on succinct rank and select indexes. The Python implementation in section 4 uses explicit arrays and therefore requires more memory. Its position mappings and query paths remain the same.

4 Simplified Implementation

The theoretical wavelet tree uses succinct indexes for the bit vectors. The implementation in this report uses explicit Python arrays. This design makes each position mapping easy to inspect and test.

Each internal node stores its alphabet range, local subsequence, and bit vector B_v. It also stores two prefix sum arrays. For $b \in \{0, 1\}$, the corresponding prefix sum array is defined by

$$R_v^b[0] = 0, \qquad R_v^b[i] = R_v^b[i-1] + \begin{cases} 1, & B_v[i] = b, \\ 0, & B_v[i] \neq b. \end{cases} \tag{13}$$

It follows directly that $\text{rank}_b(B_v, i) = R_v^b[i]$. A rank query on one node can therefore be answered with one array access.

Each node also stores the positions of its zero bits and one bits. The j-th entry of the corresponding position array gives $\text{select}_b(B_v, j)$. These arrays allow the select algorithm to move from a child sequence to its parent sequence in constant time.

The tree is constructed recursively. At each internal node, the alphabet is divided into two equal parts. Every character in the local subsequence is assigned to one child according to its next code bit. The characters are appended in their original order. This preserves the stable partition property used by the query algorithms.

The public methods use one based positions, consistent with the notation in this report. The method `access(i)` reads the current bit and moves to the corresponding child. The method `rank(c, i)` follows the binary code of c and updates the prefix length at each level. The method `select(c, j)` begins at the leaf for c and uses the stored position arrays while moving toward the root.

Every character is processed once at each of the $\log_2 \sigma$ levels. The construction time is therefore $O(n \log \sigma)$. The explicit arrays contain $O(n \log \sigma)$ integer entries in total. Python lists and integers introduce additional memory costs, so the implementation uses more space than the theoretical representation in theorem 3.1.

Each query visits one node at every level, and the work performed inside each node takes constant time. The query time is therefore $O(\log \sigma)$. Table 2 summarizes the difference between the theoretical representation and the Python implementation.

Table 2: Comparison of the theoretical representation and the implementation

Property	Theoretical representation	Python implementation
Rank support	Succinct index	Prefix sum arrays
Select support	Succinct index	Position arrays
Stored space	Near $n \log_2 \sigma$ bits	$O(n \log \sigma)$ integers
Query time	$O(\log \sigma)$	$O(\log \sigma)$

The complete Python source code is submitted as a separate supplementary file. The experiments in section 5.1 use this implementation.

5 FM Index and Experiments

The FM index performs pattern counting through backward search [2]. It stores the Burrows Wheeler transform of a text and answers each search step with rank queries.

The Burrows Wheeler transform was introduced as part of a lossless data compression method [1]. Let $T[1..n]$ be a text that ends with a unique terminator \$. The terminator is smaller than every other character. Let $SA[1..n]$ be the suffix array of T. The Burrows Wheeler transform is defined by

$$\mathrm{BW}[i] = \begin{cases} T[SA[i]-1], & SA[i] > 1, \\ T[n], & SA[i] = 1. \end{cases} \tag{14}$$

The transformed string contains the same characters as T, although their order is different. Its order reflects the lexicographic order of the suffixes.

For the running example,

$$T = \texttt{acagcagg\$},$$

the suffix array and the Burrows Wheeler transform are shown in table 3.

Table 3: Suffix array and Burrows Wheeler transform of the running example

i	$SA[i]$	Suffix	$\mathrm{BW}[i]$
1	9	`$`	`g`
2	1	`acagcagg$`	`$`
3	3	`agcagg$`	`c`
4	6	`agg$`	`c`
5	2	`cagcagg$`	`a`
6	5	`cagg$`	`g`
7	8	`g$`	`g`
8	4	`gcagg$`	`a`
9	7	`gg$`	`a`

The resulting transform is $\mathrm{BW} = \texttt{g\$ccaggaa}$.

For each character $c \in \Sigma$, define

$$C[c] = \left|\{i \mid 1 \le i \le n,\ T[i] < c\}\right|. \tag{15}$$

The value $C[c]$ is the number of characters in the text that are smaller than c. The suffixes that begin with c occupy a consecutive interval in the suffix array.

Backward search processes a pattern from right to left. Suppose that $[l, r]$ is the suffix array interval for the part of the pattern that has already been processed. When the next character is c, the new interval is

$$\begin{aligned} l' &= C[c] + \mathrm{rank}_c(\mathrm{BW}, l-1) + 1, \\ r' &= C[c] + \mathrm{rank}_c(\mathrm{BW}, r). \end{aligned} \tag{16}$$

The first rank query counts the occurrences of c before the current interval. The second rank query counts the occurrences of c up to the right boundary. The new interval contains exactly the suffixes that begin with c followed by the previously processed part of the pattern.

The algorithm begins with $[l, r] = [1, n]$. If $l > r$ after an update, the pattern does not occur. After every character has been processed, the number of occurrences is $r - l + 1$.

Consider the pattern $P = \texttt{cag}$. For the running example,

$$C[\$] = 0, \qquad C[a] = 1, \qquad C[c] = 4, \qquad C[g] = 6.$$

The interval updates are shown in table 4.

The final interval contains two suffixes. Therefore, `cag` occurs twice in the text. The two occurrences begin at positions 2 and 5.

The Burrows Wheeler transform is stored in a wavelet tree. Each rank query takes $O(\log \sigma)$ time. Since backward search processes one character of the pattern at each step, counting a pattern P takes $O(|P| \log \sigma)$ time. The array C is stored explicitly and requires $O(\sigma \log n)$ bits.

Table 4: Backward search for $P = \texttt{cag}$

Processed string	Character added	Interval
g	g	$[7, 9]$
ag	a	$[3, 4]$
cag	c	$[5, 6]$

The course also presents a more advanced representation in which rank on the Burrows Wheeler transform takes $O(\log \sigma / \log \log n)$ time. This report uses the basic wavelet tree analyzed in theorem 3.1. Its pattern counting time is therefore $O(|P| \log \sigma)$.

The supplementary Python files implement the Burrows Wheeler transform, the cumulative frequency array, and backward search. All rank queries are answered by the wavelet tree described in section 4.

5.1 Experimental Results

The experiments evaluated the correctness and running time of the Python implementation. All tests were run on the author's MacBook using Python 3. The random number generator used the fixed seed 20260724, so the same test data can be generated again.

Correctness was tested first. The access, rank, and select operations were compared with direct operations on the original string. The test included the running example and 50 randomly generated strings with lengths between 10 and 80. For each random string, the FM index was also tested with 50 random patterns. Every answer produced by the Wavelet Tree and the FM index agreed with the corresponding answer from direct scanning.

The performance test for the Wavelet Tree used strings over the alphabet $\{\$, a, c, g\}$. For each text length, 10000 access queries, 10000 rank queries, and 10000 select queries were generated. The reported value is the average time for one query.

Table 5: Construction and query times of the Wavelet Tree

n	Construction (ms)	Access (μs)	Rank (μs)	Select (μs)
1000	0.350	0.471	0.391	0.430
5000	1.576	0.848	0.420	0.456
10000	3.157	0.471	0.373	0.419
50000	15.820	0.650	0.455	0.474
100000	30.523	0.936	0.574	0.592

Figure 2 shows that the construction time increased almost linearly with the text length. When n increased from 1000 to 100000, the construction time increased from 0.350 milliseconds to 30.523 milliseconds. This behavior is consistent with the $O(n \log \sigma)$ construction time. Since the alphabet size was fixed at four, $\log_2 \sigma = 2$ was constant throughout the experiment.

The query times remained below one microsecond for every tested text length. Some variation is visible because the measured operations are very short and are affected by normal timing noise. No clear linear increase with n appears. This agrees with the theoretical query time $O(\log \sigma)$, which depends on the alphabet size and does not depend on the text length.

The FM index experiment used random patterns of length six. For each text length, 500 patterns were counted with the FM index and with direct scanning. The answers were checked again before the running times were recorded.

The average FM index query time remained close to 5.4 microseconds as the text length increased. The pattern length and alphabet size were fixed, so this result agrees with the expected

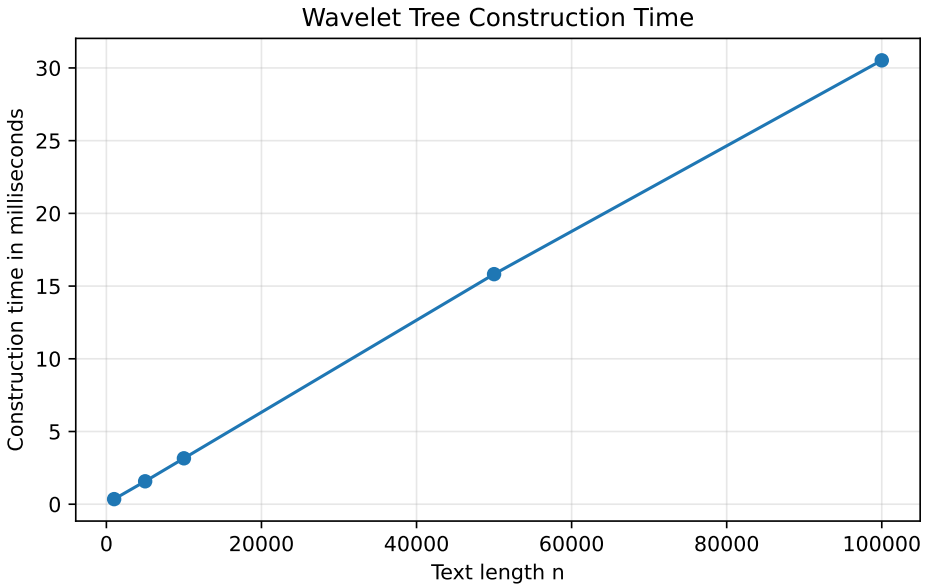


Figure 2: Construction time of the Wavelet Tree

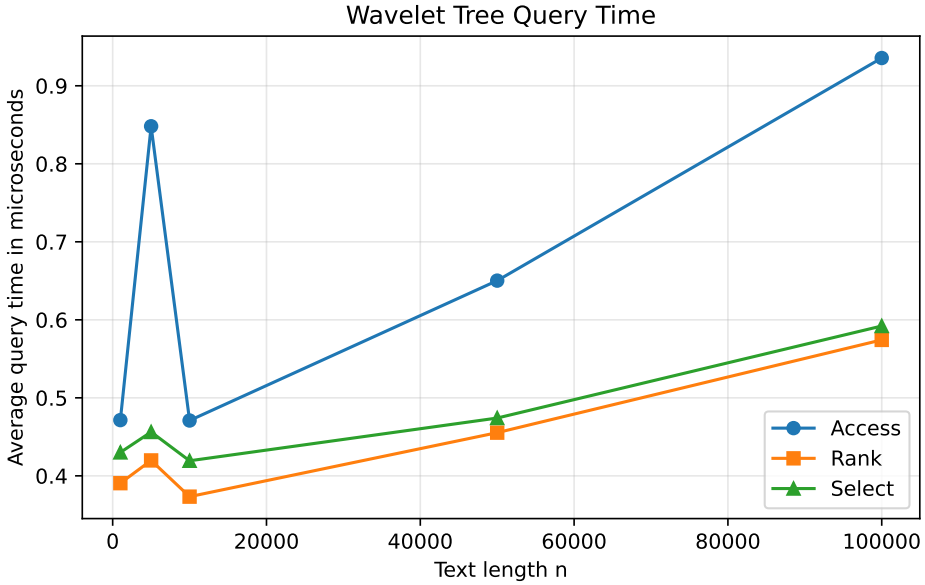


Figure 3: Average query times of the Wavelet Tree

Table 6: Pattern counting times

n	Construction (ms)	FM index query (μs)	Direct scanning (μs)
500	0.491	5.107	29.537
1000	0.769	5.470	61.421
2000	1.623	5.473	118.598
4000	5.219	5.432	242.530

time $O(|P| \log \sigma)$. Direct scanning became slower as the text length increased. Its average time rose from 29.537 microseconds at $n = 500$ to 242.530 microseconds at $n = 4000$.

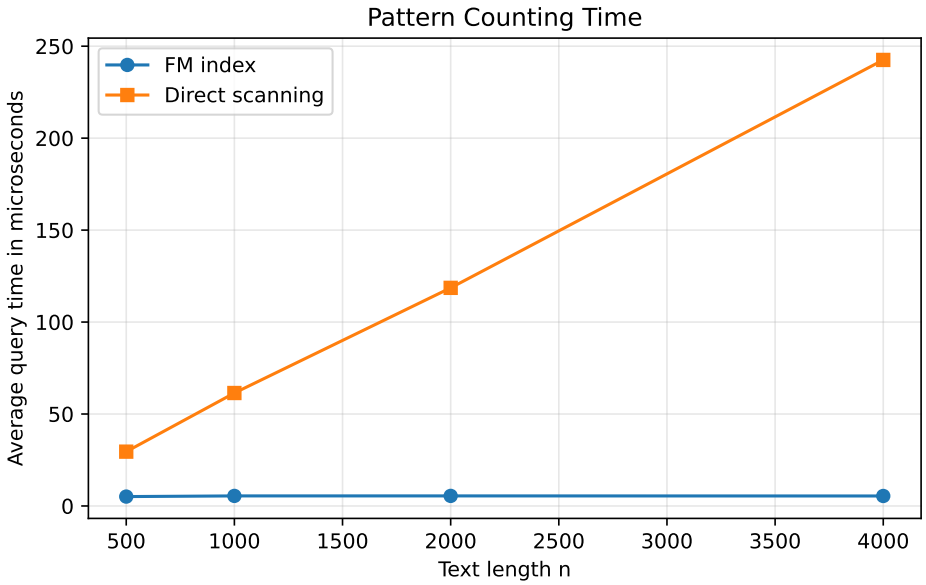


Figure 4: Comparison of FM index counting and direct scanning

At $n = 4000$, the FM index counted a pattern about 44.7 times faster than direct scanning. This comparison shows the main benefit of the index. Its query time is determined by the pattern length and the alphabet size, while direct scanning must examine the text.

The FM index construction used a simple suffix array implementation that sorts complete suffix strings. This method was sufficient for the small experiments in this report. Its construction results do not represent the performance of an efficient suffix array algorithm. The main purpose of the experiment was to verify backward search and compare its query time with direct counting.

6 Conclusion

This report studied the Wavelet Tree as a compact representation of strings over a finite alphabet. The structure divides the alphabet recursively and stores a bit vector at each internal node. These bit vectors allow access, rank, and select queries to move between successive levels of the tree.

The main theorem shows that a Wavelet Tree for a string of length n over an alphabet of size σ can be represented in $(n + o(n)) \log_2 \sigma + O(\sigma \log n)$ bits. Under the assumption that rank and select on each bit vector take constant time, all three string queries take $O(\log \sigma)$ time. The proof follows from two main facts. The total length of the bit vectors at each level is exactly n, and every query visits one node at each of the $\log_2 \sigma$ levels.

A simplified Python implementation was developed to demonstrate these operations. Each internal node stores explicit prefix sum arrays and position arrays. This representation requires more memory than the theoretical structure, although it follows the same traversal rules and preserves the $O(\log \sigma)$ query time.

The implementation was also applied to pattern counting with the FM index. The Burrows Wheeler transform was stored in a Wavelet Tree, and backward search used its rank operations. The correctness tests showed that all Wavelet Tree queries and FM index counts agreed with direct methods.

The performance results were consistent with the theoretical analysis. Wavelet Tree construction time increased approximately in proportion to the text length. Query times remained nearly

constant when the alphabet size was fixed. The FM index also maintained a stable counting time for patterns of fixed length, while direct scanning became slower as the text length increased.

The experiments used a simple suffix array construction method and relatively small alphabets. A larger implementation could use a more efficient suffix array algorithm and a succinct bit vector library. These changes would provide a closer practical realization of the space bound proved in this report.

References

[1] Michael Burrows and David J. Wheeler. A block-sorting lossless data compression algorithm. Technical Report 124, Digital Equipment Corporation, 1994.

[2] Paolo Ferragina, Giovanni Manzini, Veli Mäkinen, and Gonzalo Navarro. Compressed representations of sequences and full-text indexes. *ACM Transactions on Algorithms*, 3(2), 2007.